

Understanding the Total Loss Equation

How uncertainty weighting and L0 sparsity jointly grow an intersection circuit through gradient descent

To truly understand how this system organically grows an intersection circuit, we have to look at the Total Loss equation not as a static formula, but as a dynamic tug-of-war that happens during backpropagation. Here is the complete, unified equation governing the network:

Task A Balanced Demand	Task B Balanced Demand	The L0 Economy (Supply / Cost)
$1 / (2\sigma_A^2) \cdot \text{Loss}_A + \log(\sigma_A)$	$1 / (2\sigma_B^2) \cdot \text{Loss}_B + \log(\sigma_B)$	$\lambda_{\text{core}} L_{\text{core}} + \lambda_{\text{specific}} L_{A_only} + \lambda_{\text{specific}} L_{B_only}$

$L_{\text{Total}} = [\text{Task A term}] + [\text{Task B term}] + [\text{L0 Economy term}]$ — each bracket wrapped in its own fraction bar as shown above, all summed together.

Let's break this down completely, focusing exactly on how the math translates into physical behavior during the Backward Pass (`.backward()`), where the gradients actually build the circuits.

Part 1 — The Task Losses (The “Demand” for Nodes)

Before the scaling happens, we have the raw Loss_A and Loss_B . Remember that during the forward pass, the model created two temporary, effective circuits using the logical OR:

$$G_A = g_{\text{core}} \vee g_{A_only}$$

$$G_B = g_{\text{core}} \vee g_{B_only}$$

If Loss_A is high, it means the network failed the task when using circuit G_A . When we calculate the gradients (the derivative of the loss with respect to the gates), a high Loss_A generates a massive positive gradient signal to the gates that caused the failure.

Translation: “I need these nodes to stay OPEN to get the right answer!”

Because G_A is made of both g_{core} and g_{A_only} , both of those buckets receive this positive “keep open” signal from Task A.

Part 2 — The Uncertainty Weighting (The Equalizer)

If we just used raw Loss_A and Loss_B , a task with inherently larger numbers would scream louder and hoard all the nodes. Here is the mathematical magic of:

$$1 / (2\sigma_A^2) \cdot \text{Loss}_A + \log(\sigma_A)$$

When you run `.backward()`, the PyTorch optimizer takes the derivative of this equation with respect to σ_A and sets it to zero to find the minimum. If you do the calculus, the absolute minimum of that equation occurs when:

$$\sigma_A^2 = \text{Loss}_A$$

If you substitute that back into the equation:

$$\text{Loss}_A / (2 \cdot \text{Loss}_A) = 1/2$$

This is the crucial takeaway: no matter how massively huge Loss_A gets, or how tiny Loss_B gets, the network dynamically updates σ so that the effective gradient power they push backward into the gates is perfectly normalized (acting exactly like a weight of 0.5 for both). Task A and Task B now have the exact same gradient purchasing power.

Part 3 — The Sparsity Penalties (The “Cost” of Nodes)

While the Task Losses are screaming “KEEP OPEN!”, the Sparsity Penalties are a relentless, constant gravitational force screaming “CLOSE EVERYTHING!”

Every single parameter in L_{core} , $L_{\text{A_only}}$, and $L_{\text{B_only}}$ receives a constant negative gradient signal. The strength of that negative signal is determined exactly by your λ parameters (e.g., $\lambda_{\text{core}} = 0.6$, $\lambda_{\text{specific}} = 0.5$).

Part 4 — The Backward Pass (The Tug-of-War)

Now, let's look at a single, specific node — let's call it Node i . During the `.backward()` step, PyTorch calculates the total gradient for Node i inside the g_{core} bucket. The update for $\alpha_{\text{core},i}$ (the value that dictates if the gate is open) is the sum of three forces:

$$\Delta\alpha_{\text{core},i} = (\text{Gradient from Scaled Loss}_A) + (\text{Gradient from Scaled Loss}_B) - \lambda_{\text{core}}$$

Pushes UP (max 0.5)

Pushes UP (max 0.5)

Pushes DOWN (0.6)

Let's look at exactly how this sum determines where the node ends up.

Scenario 1 — Node i is an “A-Only” Node (Greater-Than Logic)

Task A needs this node. Task B does not care about this node.

Force	Source	Direction	Value
Force 1	Loss A	UP	+0.5
Force 2	Loss B	—	0.0
Force 3	λ_{core}	DOWN	−0.6

Total Force on g_{core} : $0.5 + 0.0 - 0.6 = -0.1$

Because the total force is negative, gradient descent will push this node out of g_{core} . It shrinks to zero. But watch what happens in the specific bucket $g_{A_{\text{only}}}$ simultaneously:

Force	Source	Direction	Value
Force 1	Loss A	UP	+0.5
Force 3	$\lambda_{\text{specific}}$	DOWN	−0.5 (approx.)

Because the node is cheaper in $g_{A_{\text{only}}}$, Task A's gradient is strong enough to keep it open there. The node correctly settles in the A-only bucket.

Scenario 2 — Node i is an “AND” Node (The Aggregator)

Task A needs this node to pass. Task B also needs this node to pass.

Force	Source	Direction	Value
Force 1	Loss A	UP	+0.5
Force 2	Loss B	UP	+0.5
Force 3	λ_{core}	DOWN	−0.6

Total Force on g_{core} : $0.5 + 0.5 - 0.6 = +0.4$

Because the total force is positive, the log-alpha value skyrockets. The gate locks wide open.

Why doesn't it go into the specific buckets? Because opening it in both $g_{\text{A_only}}$ and $g_{\text{B_only}}$ would incur a total penalty of $0.5 + 0.5 = 1.0$. The optimizer calculates that the shared g_{core} penalty of 0.6 is a steep discount, and routes the positive gradients entirely into the core bucket.

Summary of the Mechanism

You do not have to write any code that says “if in A and B, put in Core.” Because the σ terms mathematically castrate any task that tries to overpower the system, every task pushes with exactly the same strength.

Therefore, the only mathematical way a node can survive the heavy -0.6 gravity of λ_{core} is if it is actively lifted up by the combined $+0.5$ and $+0.5$ gradients of both tasks simultaneously.

That is how a single loss equation perfectly computes a logical intersection via gradient descent.

Follow-up: How Do the Downward (L0) and Upward (Scaled Loss) Forces Actually Balance?

This is the deepest level of how gradient descent actually interacts with L0 regularization. To understand exactly how these forces balance and how we know what is “enough,” we have to stop looking at the absolute loss numbers and start looking at the slopes (gradients).

The PyTorch optimizer does not care what the total loss is. It only cares about the derivative of the loss with respect to the gate parameter ($\log\alpha$). Here is exactly how the upward force (Task Loss) and the downward force (L0 Sparsity) generate specific numerical slopes, and how they fight it out.

1. The Downward Force: The L0 Derivative

Every single gate parameter ($\log\alpha$) has a continuous, mathematical downward pressure applied to it every single step. The formula for the sparsity penalty of one gate is:

$$\text{Penalty} = \lambda \times \text{Sigmoid}(\log\alpha - \text{constant})$$

When PyTorch runs `.backward()`, it takes the derivative of this penalty. The derivative of a Sigmoid function is well known: it forms a bell-shaped curve.

- The downward force is strongest when the gate is “undecided” (near 0.5 probability).
- The absolute maximum strength of this downward push is dictated entirely by λ .

If you set $\lambda = 0.6$, the maximum downward gradient force the optimizer will ever apply to close that gate is literally tied to that 0.6 scalar.

Translation: “I will constantly push this gate closed with a force proportional to 0.6.”

2. The Upward Force: The Task Loss Derivative

The task loss pushes the other way. Let's look at the scaled Task Loss:

$$\text{Scaled_Loss}_A = \text{Loss}_A / (2\sigma_A^2)$$

During the forward pass, the model used a gate (let's say it was open at 0.1 probability). During the backward pass, the calculus asks a very specific question:

“If I had opened this gate just a tiny bit more (from 0.1 to 0.11), how much would my Scaled_Loss_A have improved?”

- If opening the gate doesn't change the answer at all, the upward force is 0.0.
- If opening the gate drastically fixes a wrong answer, the slope is extremely steep, generating a massive upward force (e.g., +5.0).

3. The Tipping Point (The Balance)

The balance is simply the sum of these two exact numbers:

$$\Delta \log \alpha = (\text{Upward Task Slope}) - (\text{Downward L0 Slope})$$

How do we know if it is “enough”? It is “enough” to push the gate UP if the gate is saving more Task Loss than it costs in Sparsity Penalty.

Let's use our numbers: $\lambda_{\text{specific}} = 0.5$ and $\lambda_{\text{core}} = 0.6$.

Scenario A — A “Weak” Node (Not Essential)

Opening the node slightly helps the network's confidence, giving an upward slope of +0.2. The network tries to put it in g_{A_only} (downward slope -0.5).

Total force: $+0.2 - 0.5 = -0.3$

Result: It is NOT enough. The downward force wins. The gate closes permanently.

Scenario B — A “Strong” A-Only Node

The node is absolutely critical for Task A. If it closes, the answer is wrong. Upward slope: +3.0.

- Considering g_{core} : downward slope -0.6 $\rightarrow$ total +2.4
- Considering g_{A_only} : downward slope -0.5 $\rightarrow$ total +2.5

Result: Both are positive! The upward force easily crushes the L0 penalty. But because gradient descent always takes the steepest path to the lowest loss, it chooses the path that leaves it with the highest positive momentum (+2.5). It successfully routes to g_{A_only} .

Scenario C — The “AND” Node

- Task A needs it (upward slope +3.0)
- Task B needs it (upward slope +3.0)

If placed in specific buckets (g_{A_only} and g_{B_only}), the total downward penalty is $-0.5 - 0.5 = -1.0$. Total net momentum: $+6.0 - 1.0 = +5.0$.

If placed in the shared core (g_{core}), the downward penalty is only -0.6. Total net momentum: $+6.0 - 0.6 = +5.4$.

Result: The combined upward force (+6.0) is massive. It easily overcomes the -0.6 downward force. The optimizer picks g_{core} because +5.4 is mathematically a faster route to minimizing the total equation.

Why the Homoscedastic Weights (σ) Make This Work

Without the σ equalizer, the upward slope from a massive loss could be wildly unpredictable (e.g., Task A generates a +500.0 slope, Task B generates a +0.01 slope). If Task A has a +500.0 upward slope, it simply doesn't care if the downward λ penalty is -0.5 or -0.6. It will blow past both of them and put nodes wherever it wants.

Because σ scales the raw losses down dynamically, it ensures that the maximum upward slope a task can generate is bounded and predictable. It forces the upward gradients into a tight numerical range (e.g., between 0.0 and 5.0).

Because the upward forces are tamed and normalized by σ , your λ values (0.5 and 0.6) remain perfectly scaled to act as the strict, decisive threshold for whether a gradient is “enough” to keep a node open.